

React JS

— A powerful js library used for building

- fast
- scalable front-end applications

— created by facebook / meta

— it is a component based

— single page application (SPA's)

— virtual DOM

↳ Enabling efficient UI updates and seamless user experience. (UX).

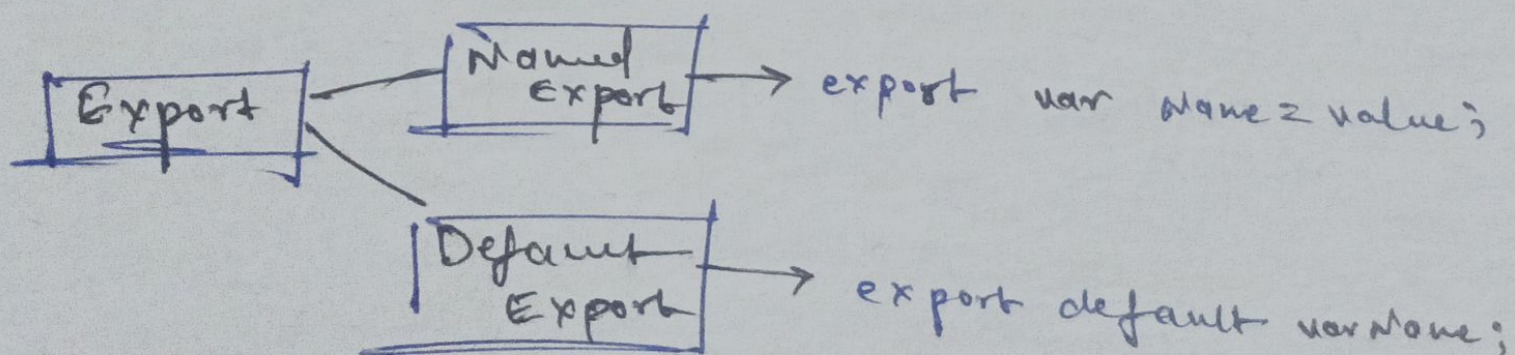
← Import and Export [JS basics]

→ Index.html — with the js file [script]

→ other js file [having to export from]

↳ for script.js [with html]

— we have to set type = "module".



Import - Named

Import { varName } from './location';

↑
Should be the exact same name.

Import - Default

import name from './location'

↑
could be any name

you can export any one default in single js file.

- Virtual DOM

React creates an copy of the actual dom called virtual dom.

↳ it updates only that portion in the whole dom which is updated.

↓ normally the whole dom tree/page used to be updated.

- JSX

→ HTML + js

- fileName.jsx

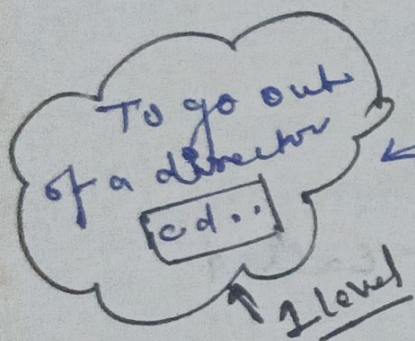
var h1 = <h1> Hello </h1>

Example

→ Component is created inside the jsx file instead of html

React Setup

- first you must have installed node.
- then in your project folder
 - ↓ in the terminal window
 - npm create vite
 - ↓
 - It will ask few questⁿ.
- If node-modules is not installed by default then go to the new created vite folder by



cd vitefilename

↳ then npm i

← To run

- npm run dev

↳ then go through local host.

Basic files structure

```
index.html — CSS
  ↓
main.jsx — CSS
  ↓
App.jsx — CSS
```

App.jsx

Here a function named App() is created
with returning only 1 component.

— But if you want to return multiple component
use

```
<>
  <div>
  <div>
</>
```

} multiple components
can be created.

— it is exported as default.

↓ And used by main.jsx

which runs this App.jsx to index.html
via a root document element

All the .jsx files has this same structure

— only 1 function and Return

[Although multiple
tasks can be done
but this is the
basic structure]

```
const App = () => {
  return (
    <>
      <Navbar />
      <div id="div hai">
      </div>
    </>
  )
}
```

Components

We create a Components folder

↳ inside which create different
jsx files for different components

Say — card
— navbar
— footer
— et — — —

→ These components has

basic jsx structure

— and have html elements

and we also create their css
files differently.

→ Files should be imported and exported
correctly for use.

↪ After the component is created

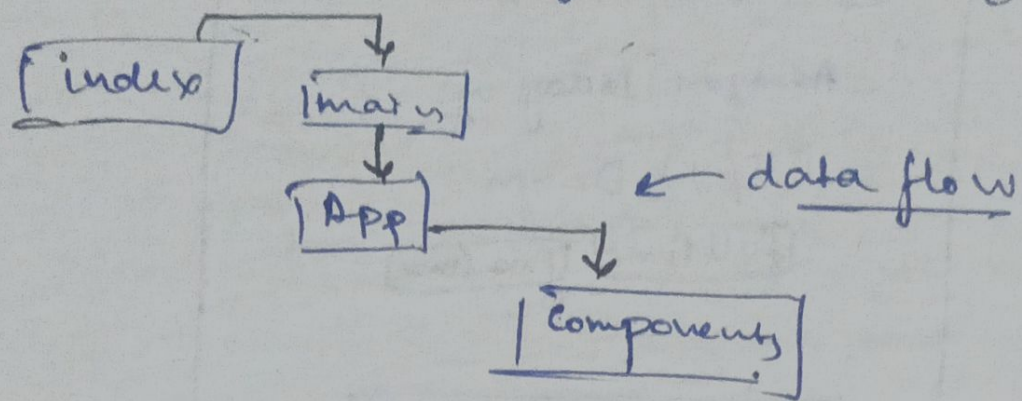
↳ it is used in the App.jsx

— in order with the different elements
as you want.

— `<Card />`

props

In react, data can only flow from parents to childrens.



Whole create card is an App.jsx.

We pass some key value ~~pair~~ pairs that would be used in the card.

<Card name="umesh" age={20} />

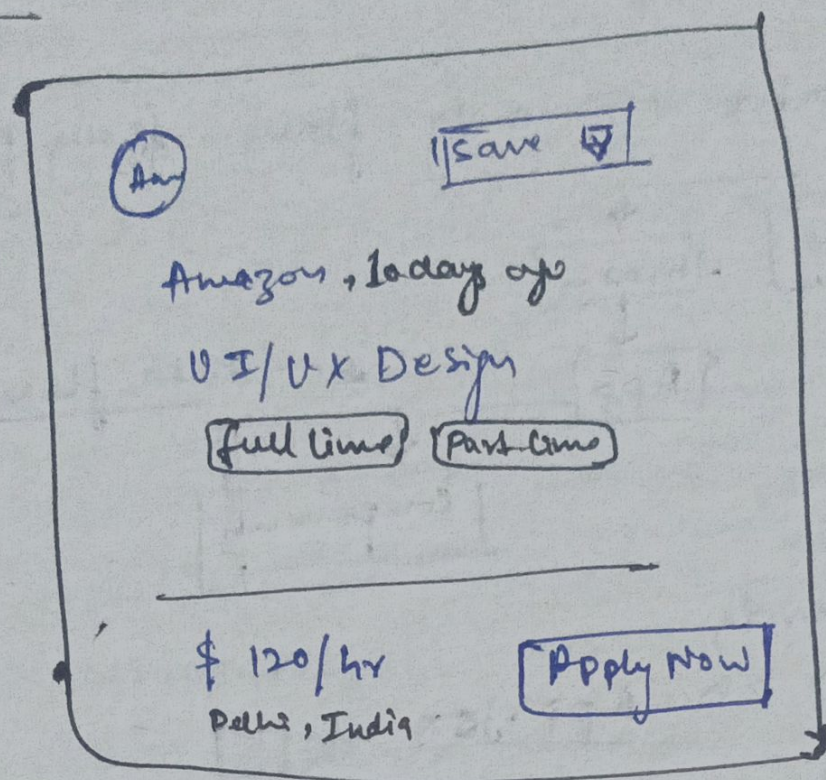
↑ Similarly multiple cards can be ~~created~~ given info for their contents

Using props in the Component

const Card = (props) => {
 return (
 <div>
 <h1> Name is {props.name} </h1>
 <h1> Age is {props.age} </h1>
 </div>
)
}

props is passed as an argument.

Card project



this is a Card — its design (html) is written in Card.jsx.

— in that function props are received

— App.jsx sends the props [key: value pairs]

→ Information is stored in an array with objects.

```
var array = [
```

```
  { name: Umesh,
    age: 20
```

```
  },
```

```
  { ————,
    ———— },
```

```
  { ————,
    ———— },
```

```
]
```

← its a json format.

Array Mapping

arr.map(function name(e){

our card
content

})

the card content
can use the objects
in the array via key:value
pair structure.

→ It has an idx for each object [w/o.s from
0 - n]

— mapping returns a new array
— unlike foreach loop.

→ mapping is used for data transformation.

— when we want to
run operations on the array

— mapping of an array is mutable.

CSS in React

— use different CSS files for different components.

↳ otherwise it can conflict.

module format

↳ put each component in a separate folder along with its CSS file

↓ the CSS file should be named as
card.module.css

— usual CSS file import

```
import 'btn.css'
```

— for module type CSS

```
import styles from './card.module.css'
```

How to use

```
<div className={styles.btn}>
```

button

```
</div>
```

btn props/CSS
will be used

for each HTML component style should be added in className.

Tailwind Setup

After creating the react project

→ go into your file in terminal

→ and install tailwind for vite

→ `npm install tailwindcss @tailwindcss/vite`

Single word

then

→ go to `vite.config.js`

→ Add the tailwind import line

`import tailwindcss from '@tailwindcss/vite'`

→ And add `tailwindcss()` in the plugins, alongside `react()`,
functⁿ

then

→ in `index.css`

→ add

`@import "tailwindcss";`

Usage

`<div className="bg-red-700 h-screen">`

different css properties
in class name.

`</div>`

functions and Event handling

Calling a function on an event

① `<btn onclick={btnClicked}> Button </btn>`

↑
only calls/runs
when it is clicked

— `{btnClicked()}`

↑
this will call the
function without even
clicking the btn.

② `<btn onclick={function () {`

`btnClicked();`

`}} >`

`button </btn>`

— Sending values via event function

`function (e) {`

`btnClicked(e.target);`

`}`

— And then in the `btnClicked` function,
you have to receive the value.

`const btnClicked = (val) => {`

`console.log(val);`

`}`

↑ function is using the given value `[e.target]`

↳ Event

- ① onclick
- ② onDoubleClick
- ③ onMouseEnter
- ④ _____ move
- ⑤ _____ leave

etc. many Events are there.

↳ Values we can use

- ① clientX, clientY
- ② pageX, pageY
- ③ screenX, Y
- ④ type: "mouse"

mouse move

- ⑤ deltaY
- ⑥

scrolling } deltaY - is scrolling speed

Hooks



Hooks are special functions that let you use state, lifecycle features and other react features inside functional components

→ without writing class components,

① useState → stores and updates component states

`const [state, set state] = useState (initial value);`

→ Triggers re-rendering on update

→ States updates are asynchronous.

② useEffect → handles side effects [side by side function running]
eg → API calls, DOM updates etc.

`useEffect ( () => { }, [dependencies] );`

③ useRef → Access DOM or stores mutable data

`const ref = useRef (null);`

④ useContext → Access global data without prop drilling.

`const value = useContext (myContext);`

→ Avoids props at many levels.

③ useReducer → complex state logic [like Redux]

const [state, dispatch]

= useReducer(reducer, initial value);

⑥ useMemo → Optimizes expensive calculations.

const memoValue = useMemo(() =>

compute(), [deps]);

⑦ useCallback → memoize function

const memoFn = useCallback(() => {},
[deps]);

→ prevents function recreation.

useState

first you should have installed useState.



import { useState } from 'react'.

→ const [state, setState] = useState(initial value);

↑
var name

↑
Set Variable

↑
will be updated

↑
initial value
to be set.

const [num, setNum] = useState(10)

setNum(20)

console.log(num);

→ it will console 10 at first
and then (2nd time) 20

it logs a step

[increase — decrease — etc
page]

form handling

`<form onsubmit = { } >`

`</form>`

⤴
Run a func on this + send e

function

`const formHandling = (e) => {`

`e.preventDefault()`

`}`

⤴
By default it reloads the whole page but we prevent it

Binding

first we have to preventDefault form submit.

→ then for any input → `onchange = { }`

function
that will take `{ e.target.value }`

and set it in a var [useState]

and use it anywhere needed

for value = { title }

⤴
~~set~~ use state variables

it will show the values accordingly by the changed values

via Noks App

→ we make 3 variable [useState]

↓ title — Note headings

Details — Note Details

Task — Array containing [title, Details] as an object.

← form onSubmit

— input 1 — for title [input]

— input 2 — for details [textarea]

↓ for each we are setting
the useState vars

setTitle(e.target.value)

setDetails(e.target.value)

in each input separately (on change)

✗ for value = {title}

and {details} — for details

→ write the form handling

prevent Default 17.

and set the title and details

in the Task useState var

as

const copyTask = [...task]
↑ making a copy of task
so it is mutable.

→ we do this in every function
it is used

then push title, details in it

copyTask.push({title, details})

then setTask (copyTask)

as func on that same ~~set~~ handling
function.

= Reset/empty the title and details
inputs

as setTitle ("");
setDetails ("");

→ mapping

Now apply mapping on task to display
the notes.

task.map (c, idx) => {

return <div>

</div>

Deleting feature

make a function to be called on delete btn

const deleteNote = (idx) => {

const copyTask = [...task]

copyTask.splice(idx, 1)

setTask(copyTask)

to delete from the array,

}

sets the task back again
after deletion.

the delete btn

should be inside the mapping
return [Notes]

And it should call the delete
function with
the idx

<btn onClick = { () => {

deleteNote (idx) .

} delete < { btn } .

Local Storage

local storage → stored in the local browser
untill it is deleted.

session storage → stored until the tab is closed.

→ To set data

`localStorage.setItem('key', 'value')`

→ To get data

`localStorage.getItem('key')`

→ Data is stored as a string only

So - to store arrays or objects we have
to convert it to string

↓ via

`JSON.stringify(key)`

↑
var name (array)

→ To get the string objects

↓ we use

`JSON.parse( )`

↑
`localStorage.getItem('key')`

clearing storage

local storage.clear()

sessionStorage.clear()

— other methods / operation

~ Remove particular item

localStm.removeItem('key')

set, get, remove, clear

API Calling f Axios

To get data from an API
we use fetch / axios

async function Name() {

const response = await fetch('link')

}

API link
↓

— async-await
↳ is used cause it takes bit of time (ms) to get the data.

— so the function/code won't proceed without fetching the API data

— install axios
npm i axios

— then import in the file to use

usage

async function name() {

const response = await axios.get('link')
}

Use Effect

Runs the side effects (side effects)

useEffect(function() {

funcName()

}, [a])

function to be run

the function will run
when a changes [dependency]

Gallery Project

- ① simple async-await for fetching images data from picum (API)
 - ② then store it in `setUserData [useState]`
- this functⁿ will be called at 1st load and then on btn click [prev & next]

useEffect (function () {

getData ()

}, [index])

index pagination

- ③ at first there will be empty screen displaying Loading ...
↳ And when data will be fetched from the API

it will be replaced by the img div and all

let profileUser = <h2 class= Loading ... </h2>

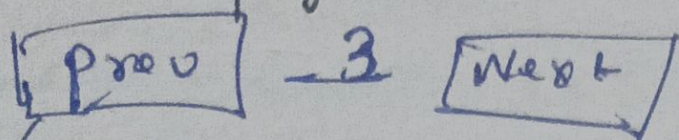
then

if (profileUser.length > 0) {

- Do mapping from API

}

④ Bty for pagination



onclick

if (index > 1) {

setIndex(index - 1)

useData[1]

}

Reduces the index taking to prev page

Empty the APP data
So new things will be stored.

for else it will do nothing (index < 1)

↓ opened by middleware

Next Bty

onclick {

setIndex(index + 1)

setUseData[1]

}

Simply so new thing data can be stored.

* How diff things will be fetched

↓ In the APP link we have

page = {{ index 3

limit = {{ youn 2 limit

gets diff things from APP data.

limit = { width }

width { }

if (window.innerWidth >= 1024) ← say 1024px

if (————— >= 768 <) ← say 768px

Router DOM

To enable routing
we have to edit main.jsx

```
ReactDOM.createRoot( ————— ) render (
  <BrowserRouter>
    <App />
  </BrowserRouter>
)
```

To use these

— install → npm i react-router-dom

import

```
import ReactDOM from ' ————— '
import { BrowserRouter } from ' ————— '
```

then in App.js or any file you want to use

import { Route, Routes } from 'react-router-dom'

<Routes>

<Route path = '/' element = {<Home/>} />

<Route path = '/about' element = {<about/>} />

</Routes>

Now, make an btn/link

instead of <a> tag
we use <link>

<link to = '/'> Home </link>

<link to = '/about'> About </link>

on click on these
it will redirect to

this first will redirect to path = '/about'
which will display the <Home/> or
<About/> component

To use {link} we have to 'import' it
from 'react-router-dom'

← Route inside a Route

Nested Routing

`<Route path = '/product' element = {<Product/>}>`

`<Route path = 'men' element = { }>`

`<Route path = 'women' element = { }>`

`<Route>`

← we used normal route tag instead of self closing tag.

← Now

Again do the same for these components in <product/>

`<div>`

`<Link to = '/product/men'> Men </Link>`

`<Link to = '/product/women'> Women </Link>`

`<div>`

`<Outlet/>`

← we have to use this for nested routing

wrong url handling

if user wrote

/courses/blank blank



incorrect page

or page doesn't exist.

the page or component you will make to display when wrong url entered.

↓ Not found.jsx

or CourseDetails.jsx

<Route path="/courses" element={<Courses />} />

Actual page → http://.../course

usi ke niche app.jsx me ye Route banana hai

Here the wrong url/text will be typing

now make the

<Route path="/courses/:anyId" element={<CourseDetails />} />

: makes it an error handler page.

anyId

→ gives the text entered on the wrong url.

could be any name

the <Course Details /> page will be showing

→ the Id : Any Id
↑ Is called useParams

It is used on the <Course Details />

How

→ import { useParams } from

'react-router-dom'.

then store it in a variable.

↪ const params = useParams()

usage

<h1> { params.anyId } </h1>

→ Handling # Main Error (url)

if at home/main site — some wrong or Non-existent (component page) text are entered.

we send the user to an NotFound.jsx (page)

<Route path = '*' element = { <NotFound /> } />

↑
Non-existing path

— props drilling — flow of data from parent
to child elements/components

Anti prop drilling

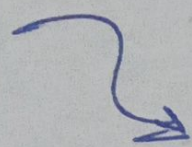
APP-JSX

make var (useState)

and pass it as props to the component

↓
both [theme and setTheme]

they will be controlled by the child
component and changes will
reflect in the APP-JSX [parent].



APP-JSX

↳ const [theme, setTheme] = useState('light')

<Navbar theme={theme} setTheme={setTheme} />

↓
and receive it in Navbar
and use (manipulate)

children as props

If you create a component with simple key
Instead of self closing tag

AppBar

```
<AppBar theme = {theme}>
```

```
<h2> Hello! </h2>
```

```
<p> lol - lol lol lol </p>
```

```
</AppBar>
```

these are childrens for
AppBar

→ so they are passed
by the same way as props
eg - {theme}

→ to relieve this is
AppBar.jsx

```
const AppBar ({children, theme}) => {
```

```
  return
```

```
}
```

you can receive it like this
or
just usual way $\rightarrow$ props

then props.children, props.theme

it will give an array
if multiple children are
present

you can access them by

children[0]

children[1] etc

use Context